

Lab 1 实验报告

姓名：肖唯远

学号：25303080005

```
sam@LAPTOP-LLA6VCIQ:~/projects/ICS-Lab-1$ ./btest
Score    Rating  Errors  Function
1         1        0      signMask
2         2        0      bitXor
3         3        0      negativePart
4         4        0      copyByteWithin
4         4        0      logicalShift
4         4        0      swapNibblePairs
4         4        0      secondLowestZeroBit
5         5        0      oddParity
5         5        0      rotateRightBits
5         5        0      roundEvenPow2
5         5        0      midpointTowardFirst
7         7        0      isBetweenEitherOrder
7         7        0      mul5Sat
7         7        0      classifyAdd3
7         7        0      floatScaleThreeHalves
10        10        0      floatRoundEven
10        10        0      float_i2f
10        10        0      bitCount
10        10        0      bitReverse
Total points: 110/110
```

```
sam@LAPTOP-LLA6VCIQ:~/projects/ICS-Lab-1$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 17/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 11/24 operators
midpointTowardFirst: 20/32 operators
isBetweenEitherOrder: 35/48 operators
mul5Sat: 23/30 operators
classifyAdd3: 32/52 operators
floatScaleThreeHalves: 54/60 operators
floatRoundEven: 30/65 operators
float_i2f: 27/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

P1 signMask

核心思路：目标是构造仅最高位为 1 的 32 位掩码。将常量 1 左移 31 位即可直接得到 0x80000000，不依赖任何输入。

P2 bitXor

核心思路：异或为“两个输入不同则为 1”。先用 $x \& y$ 找到两者都为 1 的位置，再用 $\sim x \& \sim y$ 找到两者都为 0 的位置；两类相同位置取反后，剩下的正是不同位置。

P3 negativePart

核心思路：先算术右移 x 31 位： x 为负时得到全 1 掩码，非负时得到全 0。再将 x 取补码相反数，并与该掩码相与。

P4 copyByteWithin

核心思路：把源字节右移到最低 8 位，并用 0xFF 截取。随后构造目标字节位置的 0xFF 掩码并取反，用它清空原目标字节。

P5 logicalShift

核心思路：C 的有符号右移可能进行符号扩展，因此先执行 $x \gg n$ ，再用掩码去除左侧补入的 1。掩码由最高位向右移动后再左移一位得到其反码。

P6 swapNibblePairs

核心思路：在每个字节中用重复的 0x0F 掩码分离低半字节；把它们左移 4 位。原高半字节先右移 4 位，再用相同掩码去掉跨字节带来的位。

P7 secondLowestZeroBit

核心思路：零位在 $\sim x$ 中对应为 1。令 $y = \sim x$ 后， $y \& (y - 1)$ 会去掉 y 中最低的 1，也就是去掉 x 中最低的 0。

P8 oddParity

核心思路：通过折叠异或统计全字的奇偶性：先把高 16 位异或到低 16 位，再依次按 8、4、2、1 位折叠。最终最低位等于所有原始位的异或。

P9 rotateRightBits

核心思路：循环右移由两部分组成： x 逻辑右移 n 位形成低部， x 左移 $32-n$ 位把移出的低位送回高部。

P10 roundEvenPow2

核心思路：把 x 除以 2^n 前加入偏置后再左移恢复倍数。偏置以 $\text{half} - 1$ 为基础，并在当前商为奇数时多加 1。

P11 midpointTowardFirst

核心思路：直接计算 $x+y$ 可能溢出，所以使用恒等式 $(x \& y) + ((x \wedge y) \gg 1)$ 得到不溢出的向下中点。随后仅在 x 与 y 奇偶不同的半整数场景考虑加 1。

P12 isBetweenEitherOrder

核心思路：分别判断 x 是否不小于 a 、以及是否不小于 b 。每个比较先检查符号是否不同；符号相同时用减法结果的符号判断大小。

P13 mul5Sat

核心思路：先以 $x \ll 2 + x$ 计算 $5x$ 。检查 x 左移两位后能否右移还原，以及相加时同号相加是否得到异号，二者任一成立即表示溢出。

P14 classifyAdd3

核心思路：依次计算 $a=x+y$ 和 $b=a+z$ ，并用“两个加数同号、结果异号”检测每次加法溢出。每次溢出再按加数符号分为正溢出或负溢出。

P15 floatScaleThreeHalves

核心思路：先拆分符号、阶码和尾数。NaN 与无穷大直接原样返回；有限数将有效数乘 3，再根据归一化与非规格化情况选择右移 1 或 2 位。

P16 floatRoundEven

核心思路：按阶码分区：绝对值小于 0.5 时返回带原符号的零；0.5 到 1 之间仅在严格大于 0.5 时变为 1；阶码足够大时本来就是整数。

P17 float_i2f

核心思路：先处理 0，并记录负数的符号位与绝对值。不断左移绝对值，直到最高位到达 bit 31，同时递减阶码，从而完成最高有效位定位。

P18 bitCount

核心思路：采用并行分组计数。先用 0x55 掩码按每 2 位求和，再用 0x33 掩码按每 4 位求和，接着用 0x0F 掩码把每个半字节的 1 数量聚合。

P19 bitReverse

核心思路：采用分治式交换完成 32 位逆序。先构造重复的 1 位、2 位、4 位、8 位和 16 位掩码，依次交换相邻的单个位、每 2 位组、每 4 位组、每字节以及高低 16 位。每一轮都将待交换的两部分分别移位、掩码截取后按位或，因此已完成的局部逆序会逐步扩大，最终得到整个位串的倒序结果。右移后用对应掩码清除符号扩展位，保证负数也能正确处理。